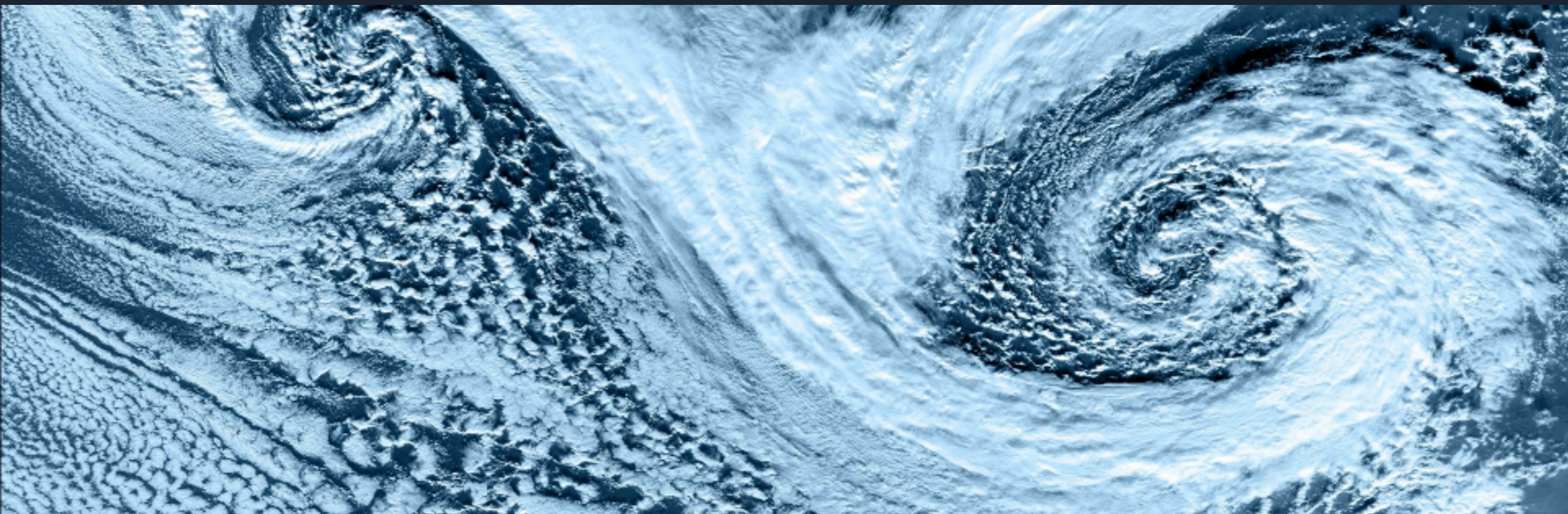
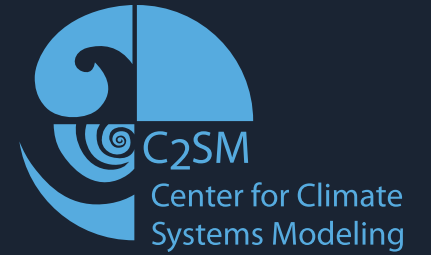


# Git: Beyond the Basics

C2SM Git Courses · 8 October 2026

Michael Jähn, Mikael Stellio, Alitzel Macías Infante



# Outline

## Part 1: Your Git Toolbox

- Examining history: `log`, `blame`, `diff`, `show`
- Nesting repositories with submodules
- Ignoring files
- `git cherry-pick` and `git rebase`
- `git stash` and `git worktree`
- Custom Git hooks
- Large files with `git lfs`
- Exercises 1 – 7

## Part 2: Working Together on GitHub

- Why a shared workflow
- A real example: the C2SM User Landing Page
- Issues, forks and branches
- Pull requests, checks and review
- Keeping your fork in sync
- GitHub and GitLab side by side
- Exercise 8

You only need the **Git basics** for this course: `add`, `commit`, `push`, `pull`, `branch`.

# Schedule

- 09:15 – 09:20**  Welcome and overview
- 09:20 – 10:40**  History, submodules, ignoring files → Exercises 1 – 3
- 10:40 – 11:00**  Coffee break
- 11:00 – 11:55**  Cherry-pick, rebase, stash, worktree → Exercises 4 – 5
- 11:55 – 12:30**  Hooks and large files → Exercises 6 – 7
- 12:30 – 13:30**  Lunch break
- 13:30 – 14:15**  Slides and live demonstration
- 14:15 – 14:55**  Exercise 8
- 14:55 – 15:00**  Wrap-up

# Part 1:

# Your Git Toolbox

# Recap: Typical Git Workflow



- Everything up to `git commit` happens **on your machine**
- Only `git push` and `git fetch` talk to the server

# Examining History: Four Commands

`git log`

which commits exist, by whom, and in what order

`git blame`

which commit last changed each **line** of a file

`git diff`

what changed between two points in the repository

`git show`

a single commit: message **and** its diff

Each takes many options. Exercise 1 explores the ones worth remembering.

# git log: Shaping and Filtering

| Option                                | What it does                                    |
|---------------------------------------|-------------------------------------------------|
| <code>--oneline</code>                | one line per commit                             |
| <code>--graph --decorate --all</code> | the branch structure of the whole repository    |
| <code>--stat</code>                   | which files each commit touched                 |
| <code>-3</code>                       | only the last three commits                     |
| <code>--author="Lauber"</code>        | only commits by a given author                  |
| <code>-- path/to/file</code>          | only commits touching that file                 |
| <code>-S 'Have fun!'</code>           | commits that <b>add or remove</b> that text     |
| <code>-G "regex"</code>               | commits whose diff matches a regular expression |

Found a combination you like? Save it: `git config --global alias.lg "log --oneline --graph --decorate --all"`

# `git diff`: Choosing Two Points

```
git diff
```

working directory vs. staging area

```
git diff --staged
```

staging area vs. last commit (what `git commit` would record)

```
git diff main my-branch
```

one branch against another

```
git diff HEAD~10 HEAD~5 -- README.md
```

two commits, restricted to one file

Hard to read in a terminal? Use `git difftool --tool-help` to see what your system offers, or the web interface: add `/compare` to any GitHub repository URL.

# Part 1 · Exercise 1

`git log`, `git blame`, `git diff` and `git show`

| Command                                                                     | What it does                           |
|-----------------------------------------------------------------------------|----------------------------------------|
| <code>git log --oneline --graph --decorate --all</code>                     | branch structure at a glance           |
| <code>git log --author="name" / -- path / -S "text"</code>                  | filter commits                         |
| <code>git blame &lt;file&gt;</code>                                         | who last touched each line             |
| <code>git diff / git diff --staged</code>                                   | unstaged vs. staged changes            |
| <code>git show &lt;commit&gt; / git show &lt;commit&gt;:&lt;path&gt;</code> | inspect a commit / an old file version |

**Where you work:** the `git-course` repository itself - we examine its real history.

All exercises: <https://github.com/C2SM/git-course/tree/main/beyond> 

# Nesting Repositories

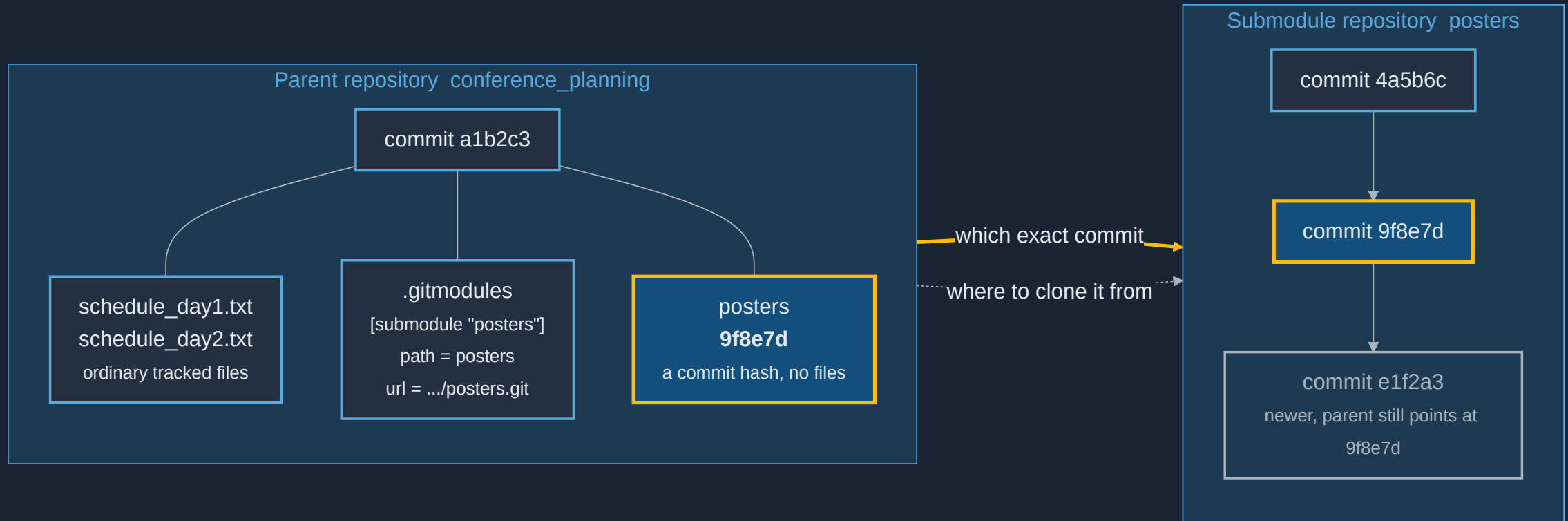
- **Why?**

- **Modularity** - break a large project into pieces that stand alone
- **Independent history** - each piece keeps its own commits and releases
- **Collaboration** - separate teams own separate pieces

- **Two mechanisms**

- **Submodules** - Git's built-in approach, and the one C2SM models use
- **Subtrees** - an alternative that copies content into the parent instead

# A Submodule Is a Pointer to One Commit



The parent repository records **one exact commit** of the submodule, not "the latest".

# Working With Submodules

```
git submodule add <url> <path>
```

adds the submodule and creates `.gitmodules`

```
git clone --recurse-submodules
```

clones the parent **and** fills in the submodules

```
git submodule update --init
```

fills them in after an ordinary `git clone`

```
git submodule update --remote --merge
```

moves the pointer forward to the submodule's latest commit

 A plain `git clone` gives you **empty** submodule directories, which is a common source of confusion.

# Submodules: Pros and Cons

## Pros

- Built into Git, nothing to install
- The parent records an exact, reproducible commit
- The submodule stays a normal repository
- Same submodule can be reused across multiple parent repos

## Cons

- Every clone needs an extra step
- Easy to commit a pointer you never pushed
- Detached `HEAD` inside the submodule confuses people
- Branch operations do not recurse by default

Some C2SM models and tools use submodules. Knowing the two or three key commands resolves most of the difficulty.

# Part 1 · Exercise 2

## git submodule

| Command                                                 | What it does                                      |
|---------------------------------------------------------|---------------------------------------------------|
| <code>git submodule add &lt;url&gt; &lt;path&gt;</code> | add a submodule                                   |
| <code>git submodule status</code>                       | which commit is currently recorded                |
| <code>git submodule update --init --recursive</code>    | fill in submodules after a plain clone            |
| <code>git submodule update --remote --merge</code>      | move the pointer to the submodule's latest commit |
| <code>git clone --recurse-submodules &lt;url&gt;</code> | clone parent and submodules in one step           |

**Where you work:** `git_beyond/conference_submodule`

Everything stays local - the helper script builds a small stand-in repository to point the submodule at.

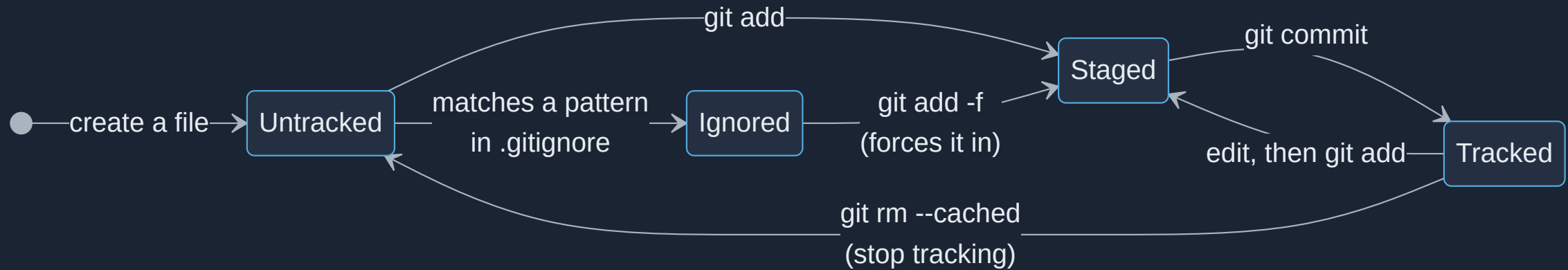
# The `.gitignore` file

- Tell Git to leave alone what should never be committed
- Build products, binaries, editor backup files, secrets, large data
- Patterns live in a `.gitignore` file - commit it, so that everyone shares the same rules

```
*~  
*.exe  
netcdf-*  
build/  
!build/keep_this.sh
```

- Patterns are matched per directory; `!` re-includes something
- `git check-ignore -v <file>` tells you **which line** ignored a file

# Ignoring Files: the States



⚠ `.gitignore` only affects **untracked** files. A file already committed keeps being tracked until you run `git rm --cached <file>`.

# `.gitkeep`

- Git tracks **files**, never directories
- An empty directory simply will not be committed
- Convention: put an empty `.gitkeep` file inside it and commit that

**Note:** `.gitkeep` is a community convention, not a Git feature. The name has no special meaning - any filename would suffice.

# Part 1 · Exercise 3

## `.gitignore` and `.gitkeep`

| Command                                                         | What it does                                              |
|-----------------------------------------------------------------|-----------------------------------------------------------|
| <code>*~, build/, !keep.sh</code>                               | pattern, directory, re-include in <code>.gitignore</code> |
| <code>git check-ignore -v &lt;file&gt;</code>                   | which rule and line ignores a file                        |
| <code>git rm --cached &lt;file&gt;</code>                       | untrack a file already committed                          |
| <code>git add path/.gitkeep</code>                              | keep an otherwise empty directory                         |
| <code>git config --global core.excludesFile &lt;file&gt;</code> | personal, machine-wide ignore rules                       |

**Where you work:** `git_beyond/conference_planning`

If you run into trouble, `reset_beyond_repo` gives you a clean start at any time.

# Coffee Break



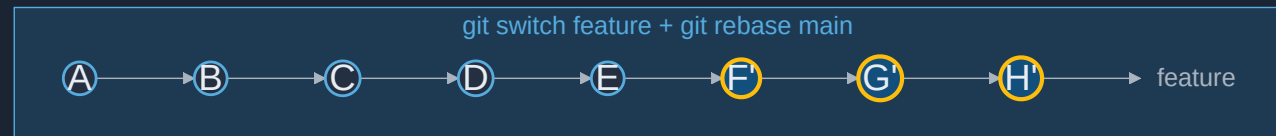
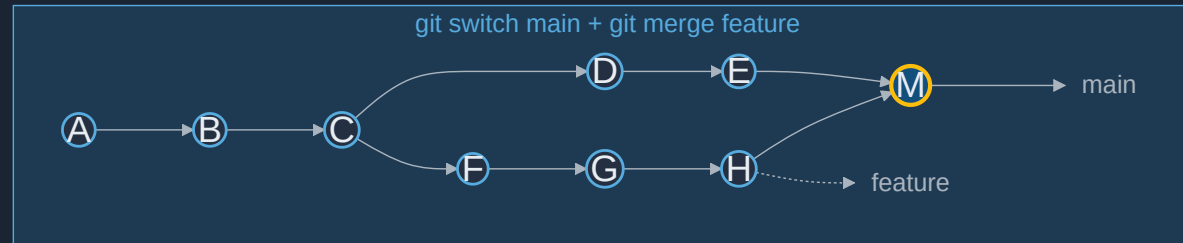
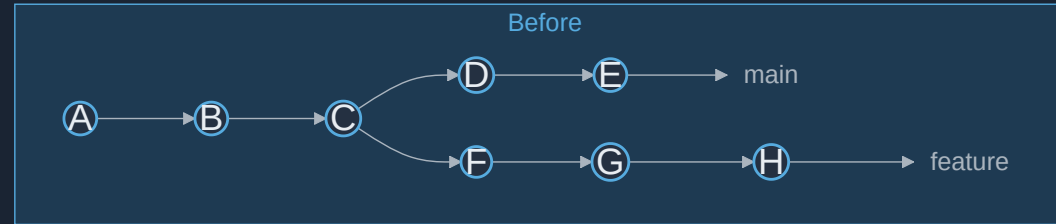
# git cherry-pick: One Commit, Copied



- Takes a single commit and replays it on your current branch
- The copy gets a **new commit ID** - same content, different identity

⚠ Do not merge the branch you picked from later: the change would arrive twice.

# git rebase: an Alternative to git merge



**Merge** joins the lines with **M** - **rebase** replays **F G H** on top of **E** as new commits **F' G' H'**

**⚠** Rebasing **rewrites history**. Never rebase a branch other people have already pulled.

# Part 1 · Exercise 4

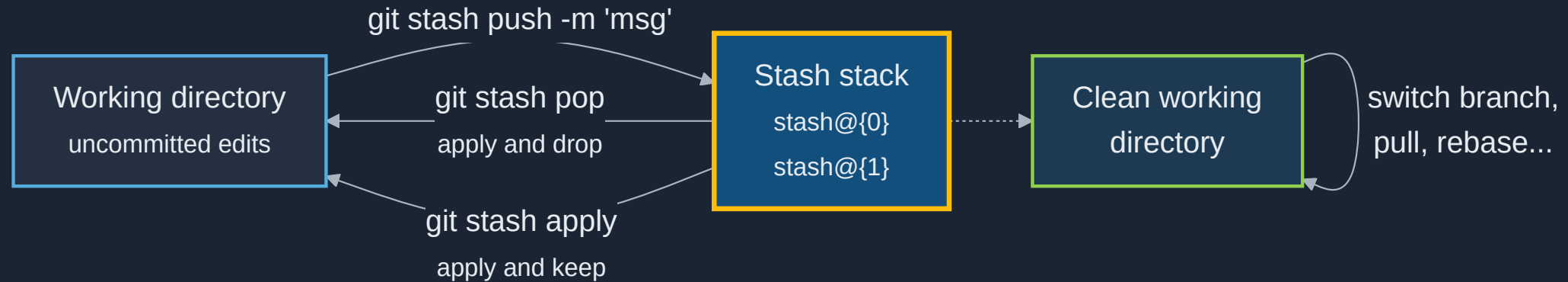
## `git cherry-pick` and `git rebase`

| Command                                                 | What it does                                  |
|---------------------------------------------------------|-----------------------------------------------|
| <code>git switch -c &lt;branch&gt;</code>               | create and switch to a new branch             |
| <code>git log --oneline --graph --decorate --all</code> | see the shape of the history                  |
| <code>git cherry-pick &lt;commit-id&gt;</code>          | replay one commit onto the current branch     |
| <code>git cherry-pick --abort</code>                    | back out of a conflicted cherry-pick          |
| <code>git rebase &lt;upstream&gt; &lt;branch&gt;</code> | replay <branch> 's commits onto <upstream>    |
| <code>git merge &lt;branch&gt;</code>                   | join two lines of history with a merge commit |

**Where you work:** `git_beyond/conference_planning`

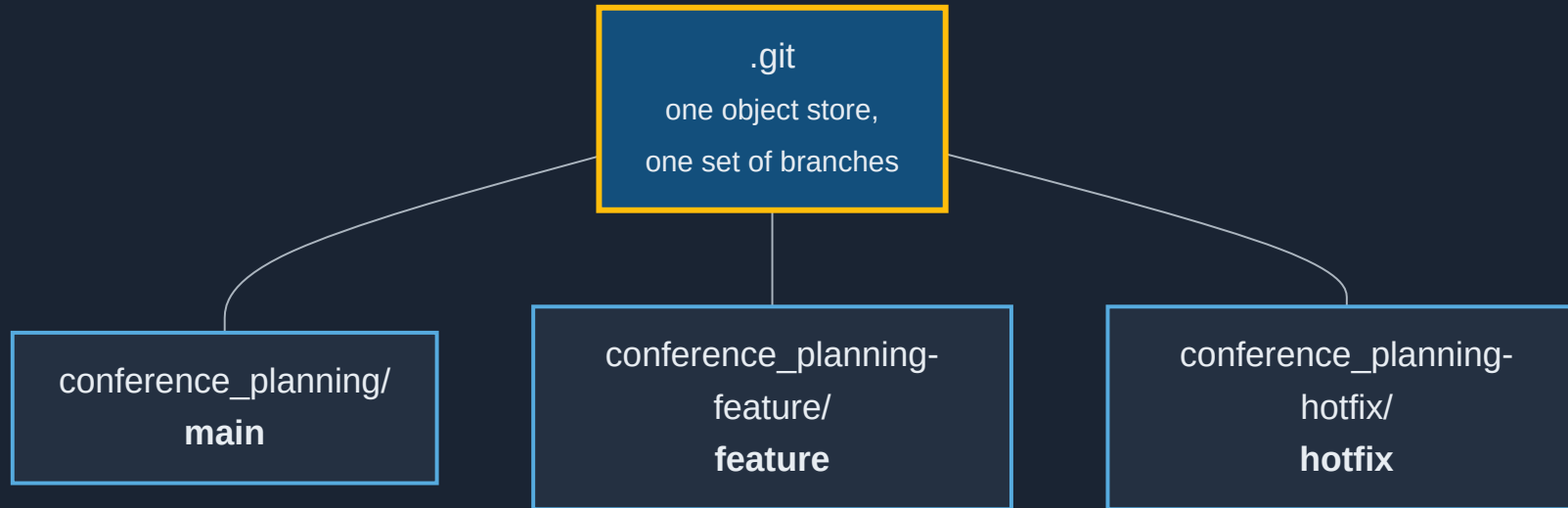
If you run into trouble, `reset_beyond_repo` gives you a clean start at any time.

# git stash: Park Your Work



- For when you must switch branch or pull, but are not ready to commit
- `git stash push -m "message"`, then `git stash list`, then `git stash pop`
- `-u` also stashes untracked files
- Stashes are **local only** - they never reach a remote

# git worktree: Several Branches at Once



- Multiple working directories sharing **one** `.git`
- Cheaper than a second clone, and the configuration stays in one place
- Especially useful when switching branches means recompiling

```
git worktree add ../conference_planning-feature feature
```

# Part 1 · Exercise 5

## git stash and git worktree

| Command                                                      | What it does                                |
|--------------------------------------------------------------|---------------------------------------------|
| <code>git stash push -m "msg" / -u</code>                    | park changes, -u includes untracked files   |
| <code>git stash list / git stash show -p stash@{0}</code>    | see stashes and what one contains           |
| <code>git stash pop / git stash apply</code>                 | restore, dropping / keeping the stash entry |
| <code>git worktree add -b &lt;branch&gt; &lt;path&gt;</code> | new working directory on a new branch       |
| <code>git worktree list / remove &lt;path&gt; / prune</code> | manage worktrees                            |

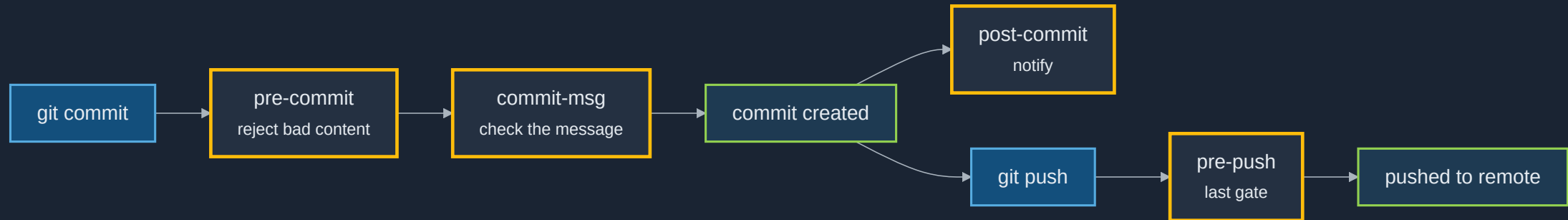
**Where you work:** `git_beyond/conference_planning` , plus the worktree it creates next to it.

# Custom Git Hooks

- Scripts Git runs automatically when a certain event happens
- Live in `.git/hooks`, named after their event, must be **executable**
- A non-zero exit status from a `pre-` hook **cancel**s the operation
- `.git/hooks` is **not** part of the repository, so hooks are not shared by cloning
- Git ships `.sample` files for every hook - rename one to activate it

Want hooks that everybody gets? Commit them to a folder and point Git at it with `git config core.hooksPath <folder>`, or use the [pre-commit](#)  framework.

# When Each Hook Fires



Typical uses: reject trailing whitespace, run a formatter, enforce a commit-message format, block commits of secrets, run fast tests before a push.

# Part 1 · Exercise 6

## Custom Git hooks

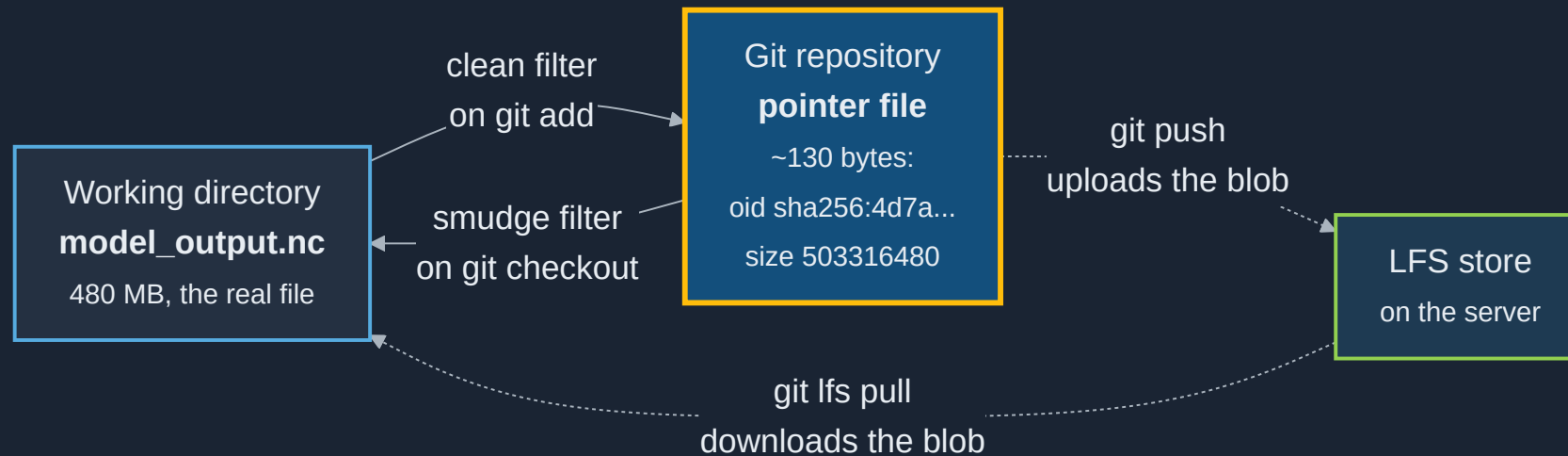
| Command                                               | What it does                                          |
|-------------------------------------------------------|-------------------------------------------------------|
| <code>.git/hooks/pre-commit</code>                    | hook file, named after the event, no extension        |
| <code>chmod +x .git/hooks/pre-commit</code>           | make it executable - required, or the hook is ignored |
| <code>git commit --no-verify</code>                   | skip hooks for this one commit                        |
| <code>git config core.hooksPath &lt;folder&gt;</code> | use a committed, shared hooks folder instead          |

**Where you work:** `git_beyond/conference_planning`

Everything happens in its `.git/hooks` directory.

# Large Files: `git lfs`

- Git stores a **full copy of every version** of every file
- That is perfect for text and terrible for a 500 MB NetCDF file
- Git Large File Storage keeps a tiny **pointer** in the repository and the real bytes elsewhere



# Using `git lfs`

```
git lfs install
```

once per machine: registers the filters

```
git lfs track "*.nc"
```

records the pattern in `.gitattributes` - **commit that file**

```
git lfs ls-files
```

which files are handled by LFS

⚠ LFS is not free: hosts put quotas on storage and bandwidth, and rewriting LFS history is painful. Before reaching for it, ask whether the data belongs in a data archive instead.

# Part 1 · Exercise 7

## git lfs

| Command                                                     | What it does                                                       |
|-------------------------------------------------------------|--------------------------------------------------------------------|
| <code>git lfs install</code>                                | once per machine: registers the filters                            |
| <code>git lfs track "*.nc"</code>                           | record a pattern in <code>.gitattributes</code> - commit that file |
| <code>git lfs ls-files</code> / <code>git lfs status</code> | which files are handled by LFS                                     |
| <code>git cat-file -p HEAD:&lt;path&gt;</code>              | see the raw pointer Git actually stores                            |

**Where you work:** `git_beyond/conference_planning`

Entirely local - no remote and no LFS quota needed.

# Lunch Break



# Part 2:

# Working Together on GitHub

# Why a Shared Workflow?

Three problems appear the moment a second person joins:

- **Access** - nobody can push to a protected `main`, so "just push it" is not an option
- **Traceability** - a change needs a visible record of *why*, not only *what*
- **Review** - somebody should look at a change **before** it reaches everyone else

A web interface solves all three with the same object: the **pull request (PR)**.

The Git commands you already know do not change. What follows is a convention layered on top of them.

# A Real Example: the C2SM User Landing Page

<https://github.com/C2SM/c2sm.github.io> → <https://c2sm.github.io>

- Documentation for models, tools, datasets and HPC systems used across C2SM
- Written as plain Markdown, built into a website automatically
- Maintained by the core team, but **anyone can contribute**
- Every pull request gets a **preview website** before anything is merged

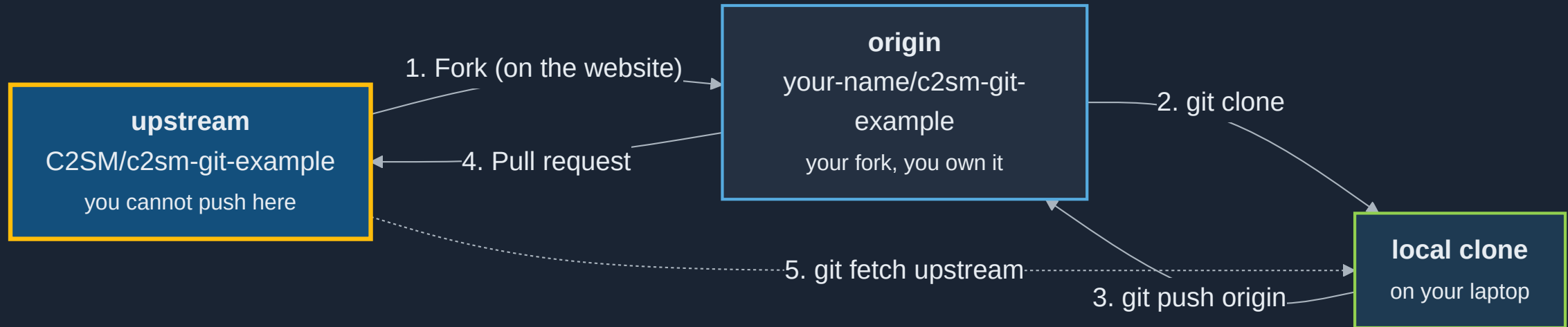
We will walk through a real change to this repository, then you will practice the same workflow on <https://github.com/C2SM/c2sm-git-example>.

# It Starts With an Issue

- An issue describes **what is wrong or missing, and why** - before any code is written
- It is the place to agree on an approach before someone spends time working on it
- Labels, assignees and milestones make a backlog searchable months later

A good issue is reproducible: what you did, what you expected, what happened instead.

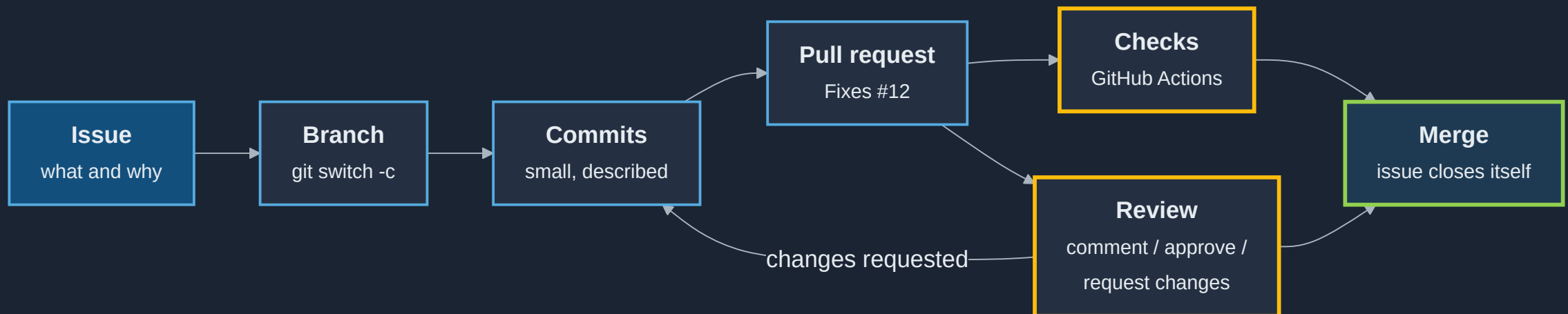
# Fork and Branch



**Fork** when you cannot push to the original. **Branch** when you can. Either way the change arrives as a pull request.

# The Pull Request

- A request to merge one branch into another, plus the conversation around it
- The **description** is the lasting record - say why, not just what
- Write `Fixes #12` in the description to link the PR and **close the issue automatically** on merge
- Open it **early** as a draft to show work in progress
- Keep it small: a reviewer reads 200 lines carefully and 2000 lines not at all



# Automated Checks

- GitHub Actions run on every pull request, before a human looks at it
- On the landing page repository they check that Markdown links resolve and that the site still builds
- A failing check blocks the merge, so broken changes never reach `main`
- The same workflow deploys a **preview of the website for that pull request**:  
`https://c2sm.github.io/pr-preview/pr-<number>/`

Reviewers can look at the rendered page, not just the diff. This is particularly useful for websites and documentation.

# Code Review

## As the author

- Small, focused changes
- Explain the reasoning
- Reply to every comment
- Push fixes as new commits

## As the reviewer

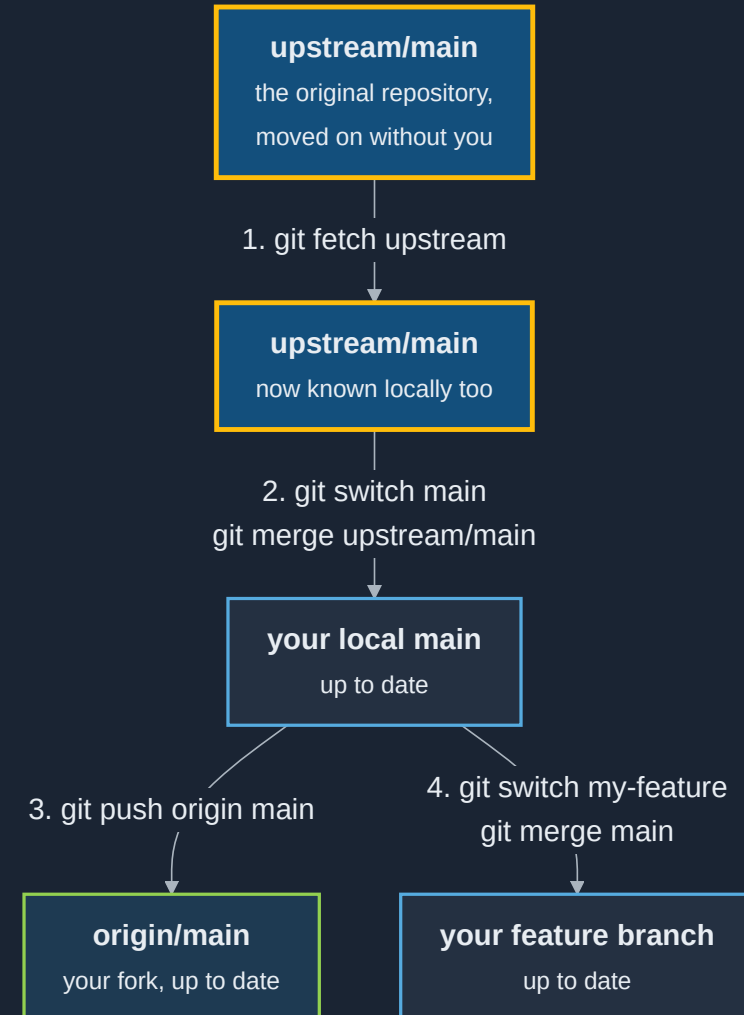
- Comment, approve, or request changes
- Use **suggestions** - the author can apply them with one click
- Ask questions instead of issuing orders
- Approve when it is good enough, not perfect

Review is about the change, never the person. Use "this function could be clearer" beats "you wrote this badly".

# Merging, and Staying in Sync

## In the GitHub PR:

- **Merge commit** - keeps every commit and records the merge
- **Squash** - collapses the branch into one tidy commit (a common default)
- **Rebase** - replays commits with no merge commit
- Delete the branch afterwards; the pull request keeps the history



# GitHub and GitLab Side by Side

| Concept                      | GitHub                                             | GitLab                              |
|------------------------------|----------------------------------------------------|-------------------------------------|
| Proposed change              | Pull request (PR)                                  | <b>Merge request (MR)</b>           |
| Close an issue automatically | <code>Fixes #12 / Closes #12</code>                | <code>Closes #12</code>             |
| CI configuration             | <code>.github/workflows/*.yml</code>               | <code>.gitlab-ci.yml</code>         |
| Sign-off on a change         | Approve / request changes                          | <b>Approvals</b> , a required count |
| Static site hosting          | GitHub Pages                                       | GitLab Pages                        |
| Preview of a change          | PR preview via an Action                           | Review Apps                         |
| Ownership rules              | <code>CODEOWNERS</code>                            | <code>CODEOWNERS</code>             |
| Update a fork                | "Sync fork" button or <code>upstream</code> remote | <code>upstream</code> remote        |
| Namespaces                   | User / organization                                | User / <b>group</b> , nestable      |

While C2SM works mostly on `github.com`, many self-hosted GitLab servers are also in use, such as `gitlab.ethz.ch` or `gitlab.dkrz.de`.

# Live Demonstration

<https://github.com/C2SM/c2sm.github.io> 

## Part 2 · Exercise 8

### An issue, a fork, a pull request and a review

| Command                                                          | What it does                                     |
|------------------------------------------------------------------|--------------------------------------------------|
| <code>git remote add upstream &lt;url&gt; / git remote -v</code> | name the original repository, list remotes       |
| <code>git switch -c &lt;branch&gt;</code>                        | start your change on its own branch              |
| <code>git push -u origin &lt;branch&gt;</code>                   | send the branch to your fork                     |
| <code>git fetch upstream / git merge upstream/main</code>        | bring your fork's <code>main</code> up to date   |
| <code>Fixes #&lt;n&gt;</code>                                    | in the PR description, closes the issue on merge |

**Where you work:** <https://github.com/C2SM/c2sm-git-example> in the browser, plus a clone of your fork anywhere outside `git_beyond`.

You will review each other's pull requests, so **work in pairs**.

# Useful Tools

## In your editor

- [VS Code](#) - built-in Git, GitLens
- [magit](#) (Emacs)
- [vim-fugitive](#) (Vim)
- [JetBrains IDEs](#)

## Better diffs

- [delta](#)
- [difftastic](#)

## In the terminal

- [gh](#) / [glab](#) - PRs and issues from the shell (GitHub / GitLab)
- [lazygit](#) - terminal interface
- [tig](#) - history browser

## Graphical

- [GitHub Desktop](#)
- [Git GUIs](#) - a long list

# Where to Look Things Up

- The Git book, free and thorough: <https://git-scm.com/book> 
- Git reference: <https://git-scm.com/docs> 
- GitHub documentation: <https://docs.github.com> 
- GitHub cheat sheet: <https://education.github.com/git-cheat-sheet-education.pdf> 
- When something has gone wrong: <https://dangitgit.com> 
- Topics beyond this course: [Expert\\_Topics.md](#) 

⚠ LLMs are often helpful for Git questions, since Git is extensively documented and widely discussed. Still, check `git help <command>` before running anything you do not recognize - especially anything with `--force`.

# References

- Chacon, S. & Straub, B. *Pro Git*, 2nd ed. Apress, 2014. <https://git-scm.com/book> 
- Git Project. *Git Reference Documentation* - `git-log`, `git-diff`, `git-submodule`, `gitignore`, `git-cherry-pick`, `git-rebase`, `git-stash`, `git-worktree`, `githooks`. <https://git-scm.com/docs> 
- Git LFS Project. *Git Large File Storage Documentation*. <https://git-lfs.com> 
- pre-commit. *A Framework for Managing Multi-Language Pre-Commit Hooks*. <https://pre-commit.com> 
- GitHub, Inc. *GitHub Docs*. <https://docs.github.com> 
- GitLab B.V. *GitLab Docs*. <https://docs.gitlab.com> 
- C2SM. *c2sm.github.io* - the User Landing Page used as the worked example. <https://github.com/C2SM/c2sm.github.io> 

# Questions? Comments?